Lab 5: Adversarial Search: Minimax & Alpha-Beta Pruning

Week 5 | Lecture(s): L11-L12 | Maps to CLO2 (C3: Apply) | 3C Focus: Competence

Objective: Apply Minimax and Alpha-Beta pruning to game trees and explain how pruning reduces search while preserving the Minimax decision.

INQUIRY-BASED LESSON PLAN - 3C MODEL

1. LEARNING OUTCOMES (COMPETENCE)

- Represent alternating MAX/MIN decisions in a game tree.
- Compute Minimax values bottom-up.
- Apply alpha and beta bounds to prune irrelevant branches.
- Compare evaluated leaves with and without pruning.

2. REAL-WORLD PROBLEM (INQUIRY START)

A game-playing agent must choose a move while assuming the opponent also acts optimally. Searching every leaf is expensive, so the agent needs a way to avoid branches that cannot change the final decision.

3. TRIGGER QUESTIONS (COURAGE - THINKING)

- Why does MAX assume MIN will choose the worst outcome for MAX?
- What do alpha and beta represent?
- Does pruning change the optimal Minimax result?
- Why does move ordering affect pruning efficiency?

4. STUDENT ENGAGEMENT ACTIVITY

Human Game-Tree Trace: students label a small tree from leaves to root, then repeat with alpha-beta bounds.

5. CONCEPT INTRODUCTION (AFTER INQUIRY)

Game trees; terminal utility; MAX/MIN layers; Minimax; alpha-beta bounds; pruning condition; move-ordering trade-offs.

6. APPLICATION TASK (COMPETENCE)

Implement Minimax and alpha-beta for a small fixed-depth tree and count evaluated leaves.

7. CHARACTER INTEGRATION

Transparency: show the tree, utilities and pruning decisions so another student can reproduce the chosen move.

8. REFLECTION

- What did you learn today?
- How will you apply this in a real AI system?
- What failed or surprised you?
- What would you do differently next time?

9. MEASURABLE OUTCOME (COMPETENCE)

Correct root value and action, plus evidence that alpha-beta evaluates no more leaves than plain Minimax for the same ordering.

10. FACULTY SELF-CHECK (END OF LAB)

- Did I start with the stated course problem and lecture outcome?
- Did students investigate or predict before the formal explanation?
- Did every student implement, trace, test, or evaluate something practical?
- Did I require evidence of testing, comparison, or debugging?
- Did the activity remain aligned with the mapped CLO and cognitive level?
- Did the collected evidence match the measurable outcome?

CASE STUDY / PRACTICAL SCENARIO

Case Study: Choose the best move in a small two-player zero-sum game tree where terminal scores are known.

Required evidence: working code/procedure, captured output, at least one edge-case or comparison test, a short interpretation of results, and a viva-ready explanation of design choices.

5.1 Minimax

```
import math

# Maximizing player can get
def minimax(curDepth, nodeIndex, maxTurn, scores, targetDepth):

    # Base case: targetDepth reached
    if curDepth == targetDepth:
        return scores[nodeIndex]

    if maxTurn:
        return max(
            minimax(curDepth + 1, nodeIndex * 2, False, scores, targetDepth),
            minimax(curDepth + 1, nodeIndex * 2 + 1, False, scores, targetDepth)
        )
    else:
        return min(
            minimax(curDepth + 1, nodeIndex * 2, True, scores, targetDepth),
            minimax(curDepth + 1, nodeIndex * 2 + 1, True, scores, targetDepth)
        )


# Driver code
scores = [3, 5, 2, 9, 0, 7]
treeDepth = math.ceil(math.log2(len(scores)))

# Pad scores to make a complete binary tree
scores += [0] * (2**treeDepth - len(scores))

print("The optimal value is:", end=" ")
print(minimax(0, 0, True, scores, treeDepth))
```

5.2 Alpha-Beta Pruning

```
import math

def alphabeta(depth, index, maxTurn, scores, target, alpha, beta):
    if depth == target:
        return scores[index]

    if maxTurn:
        value = float('-inf')
        for i in range(2):
            value = max(value, alphabeta(depth+1, index*2+i, False, scores, target, alpha, beta))
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value = float('inf')
        for i in range(2):
            value = min(value, alphabeta(depth+1, index*2+i, True, scores, target, alpha, beta))
            beta = min(beta, value)
            if alpha >= beta:
                break
        return value

scores = [3, 5, 2, 9, 3, 5, 2, 9]
depth = int(math.log2(len(scores)))

print(alphabeta(0, 0, True, scores, depth, float('-inf'), float('inf')))
```

Lab Task(s)

1. Trace the supplied tree by hand using Minimax.
2. Add a leaf-evaluation counter to both algorithms.
3. Reorder branches and observe how pruning changes while the Minimax value remains the same.

REFERENCE SOLUTION / INSTRUCTOR CHECK (CODE / PROCEDURE)

The following is a reference implementation/check. Students should understand, trace and test it rather than submit it unchanged. Equivalent solutions are acceptable when the reasoning and results are correct.

```
# Reference check: both functions must return the same root value for the same complete tree.
Instrument the terminal-node case with a counter to compare work.
```

Expected Result / Verification

Minimax and alpha-beta must agree on the root value. Alpha-beta can reduce evaluated leaves; the amount depends on ordering.

Submission Evidence

- Source code / notebook or structured working where applicable
- Visible outputs, trace tables, plots, metrics or screenshots
- Short explanation connecting the result to the lecture learning outcome
- At least one edge case, comparison, limitation or failure analysis
- Reflection and responsible-practice response